Lab 3 Rejestr kont, wyjątki i przelewy

1 Cel zajęć

Rozbudowujemy projekt bankowy: konta dostaną wpłaty i wypłaty, a bank rejestr kont i przelewy. Poznasz wyjątki i sposób testowania błędów. Przed implementacją przelewu poprosisz AI o review swoich testów.

- **Po zajęciach:** napiszesz test, który sprawdza, że kod rzuca wyjątek (`pytest.raises`),
- zbudujesz klasę z kolekcją kont i przelew między kontami,
- sprawdzisz, czy AI wskazało w Twoich testach luki, i zdecydujesz, które uwagi uwzględnić.

Kod, nazwy testów, commity i prompty piszemy **po angielsku**.

2 Start: Lab 2 w main

> **Zanim zaczniesz nowe zadania**
>
> Praca z Lab 2 musi być zmergowana do `main` (PR zamknięty, CI zielone). Jeśli nie jest, dokończ PR. Nowa praca zaczyna się zawsze od aktualnego `main`.

```
git switch main
git pull
python -m pytest              # all green?
git switch -c lab3
```

3 Wyjątki i pytest.raises

Dotąd błędne dane obsługiwaliśmy wartością (PESEL → `"Invalid"`). Przy wypłacie to nie wystarczy: zwróconą wartość łatwo przeoczyć, a operacja i tak by się wykonała. **Wyjątek** przerywa operację i mówi, co poszło nie tak. Wywołujący musi na to zareagować.

```
# src/exceptions.py
class InsufficientFundsError(Exception):
    pass

# src/account.py
def withdraw(self, amount):
    if amount > self.balance:
        raise InsufficientFundsError("Not enough funds")
```

Test sprawdza, że wyjątek faktycznie poleciał. Bez raise test jest czerwony i pytest pisze DID NOT RAISE (to nasz Red):

```
import pytest
from src.account import Account
from src.exceptions import InsufficientFundsError

def test_withdraw_more_than_balance_raises():
    account = Account("Jan", "Kowalski", "90010112345")      # balance 0
    with pytest.raises(InsufficientFundsError):
        account.withdraw(100)
    assert account.balance == 0                               # state unchanged
```

- **Wyjątek konkretny, nie** `Exception`: inaczej test przejdzie przy dowolnym błędzie, także literówce w kodzie.

- **Zły argument** (np. kwota ≤ 0) to wbudowany `ValueError`. Własny wyjątek tworzymy dla błędów biznesowych (brak środków, brak konta).
- **Po wyjątku sprawdź stan:** nieudana operacja nie może zmienić salda.

4 Zadania

Dla każdej funkcji: test, czerwony, kod, zielony, refaktor, commit. Wyjątki trzymamy w `src/exceptions.py`.

4.1 Wpłata i wypłata

Dodaj do `Account` metody `deposit(amount)` i `withdraw(amount)`. Kwota musi być dodatnia, inaczej `ValueError`. Wypłata większa niż saldo rzuca `InsufficientFundsError` (saldo bez zmian). Wypłata całego salda jest dozwolona.

> **Zrobione, gdy:** testy: wpłata, wypłata, kwota 0 i ujemna, wypłata ponad saldo, wypłata całego salda. Co najmniej 3 testy z pytest.raises, commit.

4.2 Rejestr kont

Konto nie wie o innych kontach, a przelew potrzebuje miejsca, gdzie je znaleźć. Stwórz klasę `AccountRegistry` w `src/account_registry.py`. Kluczem jest na razie PESEL.

Metoda	Zachowanie
`add_account(account)`	dodaje konto; PESEL już w rejestrze lub konto z PESEL-em "Invalid" → ValueError
`find_by_pesel(pesel)`	zwraca konto albo None, gdy go nie ma
`count()`	liczba kont
`all()`	lista kont (kopia: jej zmiana nie wpływa na rejestr)

Wskazówka: w testach rejestru powtarza się „utwórz pusty rejestr”. W Refactor zamień to na fixture (`@pytest.fixture`).

> **Zrobione, gdy:** testy: pusty rejestr, dodanie i znalezienie, brak konta (None), duplikat, niepoprawny PESEL, kopia z all(). Commit.

4.3 Przelew (z review testów przez AI)

Dodaj `registry.transfer(sender_pesel, recipient_pesel, amount)`. Wymagania: brak któregoś konta → `AccountNotFoundError` (nowy wyjątek), nadawca = odbiorca lub kwota ≤ 0 → `ValueError`, brak środków → `InsufficientFundsError`. Przy każdym błędzie **oba salda zostają bez zmian**.

1. **Testy najpierw.** Napisz testy przelewu (czerwone). **Nie implementuj jeszcze** `transfer`.
2. **Review testów.** Uruchom `claude` lub `gemini` w folderze projektu i wklej prompt:

```
Review the tests for AccountRegistry.transfer in tests/unit/test_account_registry.py.
List scenarios that are missing or weak. Do not write any code or tests,
only describe the scenarios.
```

3. **Decyzja.** Każdą propozycję oceń: **dodaję** (realna luka), **już mam**, **poza zakresem** lub **chybiona** (AI się myli). Zapisz oceny, trafią do opisu PR.
4. **Uzupełnienie testów.** Dopisz testy do co najmniej jednej propozycji, którą uznałeś za trafną.
5. **Implementacja.** Napisz `transfer` i doprowadź testy do zielonego, korzystając z `withdraw` i `deposit`, bez powielania walidacji. Zastanów się: co wykonać pierwsze, żeby błąd nie zostawił pieniędzy „w powietrzu”?

Zrobione, gdy:

- testy przelewu: sukces, cała kwota, brak środków, kwota ≤ 0, brak nadawcy, brak odbiorcy, ten sam numer
- po każdym błędzie testy sprawdzają, że salda się nie zmieniły
- masz listę propozycji AI z Twoją oceną; commit

5 Pull request

Otwórz PR `lab3 → main`. Opis wygeneruj z AI tak jak w Lab 2, sprawdź go i dopisz sekcję **AI test review**: lista propozycji z punktu 4.3, Twoja decyzja i krótkie uzasadnienie.

Zrobione, gdy:

- PR lab3 → main otwarty, opis sprawdzony i poprawiony, jest w nim sekcja AI test review,
- CI (Actions) zielone,
- merge zrobiony.

6 Gdy coś nie działa

- `DID NOT RAISE`: kod nie rzuca wyjątku. Na etapie Red to dobrze, po implementacji sprawdź warunek.
- `ImportError: cannot import name ...`: sprawdź, czy wyjątek jest w `src/exceptions.py` i czy import zgadza się z nazwą klasy.
- Test przechodzi, choć kod jest zły: sprawdź, czy `pytest.raises` nie łapie zbyt ogólnego wyjątku.

Do poczytania: wyjątki w Pythonie · pytest: oczekiwane wyjątki · pytest: fixtures.